
Sprawozdanie-z-labow

pawel

May 19, 2026

SPIS TREŚCI:

1	Wprowadzenie	3
1.1	Rozdział 1: Kontrola i konserwacja	3
1.2	Badania literaturowe	5
1.3	Rozdział 3: Projektowanie Bazy Danych: System Zarządzania Biblioteką	14
1.4	Rozdział 4: Implementacja fizyczna i zasilanie bazy danych	19

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

WPROWADZENIE

Niniejsza dokumentacja stanowi kompletne sprawozdanie z laboratorium przedmiotu Bazy Danych. Projekt kompleksowo obejmuje aspekty związane z zarządzaniem środowiskiem bazodanowym, projektowaniem struktury relacyjnej oraz implementacją fizyczną.

Dokumentacja została podzielona na cztery główne sekcje tematyczne, począwszy od teoretycznych aspektów konserwacji systemów, poprzez badania aż po wdrożenie bazy danych dla systemu zarządzania biblioteką.

1.1 Rozdział 1: Kontrola i konserwacja

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

1.1.1 Wprowadzenie

Współczesne systemy relacyjnych baz danych to wysoce skomplikowane środowiska, które wymagają ciągłego i proaktywnego nadzoru, aby zagwarantować wysoką dostępność, niezawodność oraz optymalny czas odpowiedzi na zapytania klientów. Wraz z rosnącym wolumenem przetwarzanych informacji, zarządzanie cyklem życia danych staje się wyzwaniem równie ważnym, co sam projekt struktury relacyjnej. Rozdział ten poświęcony jest nowoczesnym mechanizmom kontroli i konserwacji, ze szczególnym uwzględnieniem architektury silnika PostgreSQL. Procesy te obejmują zaawansowane zarządzanie przestrzenią dyskową, analizę i optymalizację planów zapytań, rygorystyczną kontrolę dostępu oraz wielowarstwowe strategie zabezpieczania danych przed ich bezpowrotną utratą.

1.1.2 1.1. Zarządzanie przestrzenią i mechanizm współbieżności

PostgreSQL opiera swoje działanie na zaawansowanej architekturze określanej jako Multi-Version Concurrency Control. Technologia ta pozwala na równoległy odczyt i zapis danych bez wzajemnego blokowania się transakcji, co drastycznie zwiększa przepustowość systemu przy dużej liczbie jednoczesnych połączeń. Skutkiem ubocznym tego bezkolizyjnego mechanizmu jest jednak powstawanie martwych krotek, czyli przestarzałych wersji wierszy. Wierze te, choć zostały już zaktualizowane lub usunięte przez aplikację, wciąż fizycznie zajmują przestrzeń na dysku ze względu na to, że inne, wciąż otwarte transakcje, mogą potrzebować dostępu do ich historycznego stanu.

Aby zapobiec zjawisku drastycznego puchnięcia tabel oraz degradacji wydajności operacji wejścia i wyjścia, konieczna jest regularna konserwacja na poziomie nośnika danych:

- **VACUUM:** Podstawowy proces konserwacyjny odzyskujący miejsce po martwych krotkach. Oznacza on wolne obszary jako dostępne do ponownego zapisu przez nowe instrukcje, zapobiegając niekontrolowanemu rozrostowi plików bazy.
- **VACUUM FULL:** Znacznie bardziej inwazyjna wersja operacji, która fizycznie przebudowuje strukturę całej tabeli i trwale zwraca wolne miejsce do systemu operacyjnego. Operacja ta wymaga jednak nałożenia wyłącznej blokady na modyfikowany obiekt, co uniemożliwia jego odczyt i zapis na czas trwania przebudowy.

- **AUTOVACUUM:** W nowoczesnych środowiskach produkcyjnych utrzymanie czystości dyskowej powierza się wbudowanemu procesowi pracującemu w tle. Monitoruje on na bieżąco statystyki zmian i automatycznie inicjuje procesy czyszczenia po przekroczeniu zdefiniowanych progów aktywności, co minimalizuje konieczność ręcznej ingerencji administratora.

Zaniedbanie procesu czyszczenia w systemach o wysokiej rotacji danych może doprowadzić nie tylko do spadku wydajności, ale w skrajnych przypadkach do wyczerpania identyfikatorów transakcji, co skutkuje całkowitym wstrzymaniem działania bazy.

```
-- Przykładowe wymuszenie manualnej konserwacji z analizą statystyk dla tabeli
VACUUM (VERBOSE, ANALYZE) Wypozyczenia;
```

1.1.3 1.2. Optymalizacja i statystyki planisty zapytań

Kolejnym filarem utrzymania sprawności systemu jest kontrola nad optymalizatorem, zwanym planistą zapytań. Silnik bazy danych przed wykonaniem jakiegokolwiek skomplikowanej kwerendy analizuje dziesiątki możliwych ścieżek jej realizacji. Decyzje takie jak wybór między skanowaniem sekwencyjnym całego zbioru a użyciem konkretnego indeksu podejmowane są na podstawie modelu matematycznego opartego o szacowany koszt operacji.

Aby planista mógł podejmować optymalne decyzje, musi opierać się na wysoce dokładnych i aktualnych statystykach dotyczących rozkładu danych. Obejmują one między innymi histogramy wartości, listy najczęściej występujących elementów oraz współczynniki korelacji między kolumnami.

- Instrukcja **ANALYZE** służy do natychmiastowego odświeżenia tych metadanych. Baza pobiera losową próbkę wierszy i na jej podstawie aktualizuje wewnętrzne tabele systemowe, co pozwala uniknąć katastrofalnych w skutkach błędów estymacji optymalizatora.
- Bieżące profilowanie wydajności realizowane jest natomiast przy pomocy instrukcji **EXPLAIN ANALYZE**. Narzędzie to uruchamia zapytanie, po czym zwraca nie tylko wynik, ale również szczegółowy raport z rzeczywistego przebiegu wykonania. Pozwala to programistom i administratorom precyzyjnie namierzyć wąskie gardła w skomplikowanych kwerendach, które łączą wiele tabel i filtrują ogromne zbiory danych.

```
-- Kontrola planu wykonania zapytania z rzeczywistym pomiarem czasu
EXPLAIN ANALYZE
SELECT c.Imie, c.Nazwisko, k.Tytul
FROM Czytelnicy c
JOIN Wypozyczenia w ON c.ID_Czytelnika = w.ID_Czytelnika
JOIN Ksiazki k ON w.ID_Ksiazki = k.ID_Ksiazki
WHERE w.Data_Zwrotu IS NULL;
```

1.1.4 1.3. Bezpieczeństwo i rygorystyczna kontrola dostępu

Bezpieczeństwo danych stanowi absolutny priorytet każdej administracji systemami informatycznymi. Nowoczesne podejście do ochrony baz danych całkowicie odchodzi od wykorzystywania globalnych kont administracyjnych, takich jak domyślny użytkownik instalacyjny, do codziennych operacji na poziomie aplikacji. Zamiast tego wdraża się model kontroli dostępu opartej na rolach, który znacząco ogranicza ryzyko kompromitacji całego systemu w przypadku wycieku pojedynczego poświadczenia.

System zabezpieczeń środowiska PostgreSQL pozwala na niezwykle elastyczną gradację uprawnień. Definiuje się dedykowane role systemowe przypisane do konkretnych mikrousług lub modułów aplikacyjnych. Poświadczenia dostępowe szyfrowane są przy wykorzystaniu najnowocześniejszych standardów kryptograficznych, takich jak algorytm SCRAM-SHA-256, odporny na ataki słownikowe i nasłuchiwanie ruchu sieciowego.

Kluczowym konceptem jest wdrożenie zasady najmniejszych uprawnień. Rola wykorzystywana przez aplikację powinna dysponować prawami pozwalającymi wyłącznie na modyfikację danych niezbędnych do jej funkcjonowania. W praktyce oznacza to celowe przyznawanie praw do wykonywania selekcji czy dodawania rekordów, przy jednoczesnym blokowaniu możliwości ich usuwania lub modyfikacji struktury tabel.

```
-- Przykład implementacji bezpiecznej polityki kontroli dostępu
CREATE ROLE app_user WITH LOGIN PASSWORD 'TrudneHaslo123';
GRANT CONNECT ON DATABASE biblioteka_db TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT ON Wypozyczenia TO app_user;
```

1.1.5 1.4. Strategie zabezpieczania przed utratą danych

Nawet najlepiej zoptymalizowany i zabezpieczony system informatyczny jest narażony na błędy ludzkie lub awarie infrastruktury sprzętowej. Dlatego odpowiednia polityka wykonywania kopii zapasowych oraz odtwarzania środowiska po awarii jest krytycznym elementem konserwacji relacyjnych baz danych. Zarządzanie tym procesem opiera się na dwóch uzupełniających się podejściach.

Kopie logiczne polegają na ekstrakcji danych z bazy do postaci czystego skryptu języka SQL lub dedykowanego formatu archiwalnego. Narzędzia realizujące to zadanie gwarantują przenośność danych pomiędzy różnymi wersjami silnika bazy czy architekturami sprzętowymi. Proces tworzenia kopii logicznej potrafi być jednak bardzo obciążający dla procesora, a czas odtwarzania z takiego pliku w przypadku wieloterabajtowych struktur jest często nieakceptowalnie długi z biznesowego punktu widzenia.

Dlatego w krytycznych środowiskach produkcyjnych stosuje się kopie fizyczne połączone z archiwizacją dzienników wyprzedzających. Dzienniki te rejestrują absolutnie każdą zmianę bajtu na dysku jeszcze przed jej ostatecznym zatwierdzeniem. Odpowiednio skonfigurowany mechanizm ciągłej archiwizacji tych strumieni pozwala na odtworzenie całej bazy danych z chirurgiczną precyzją, do konkretnej sekundy w przeszłości. Taka funkcjonalność pozwala cofnąć skutki przypadkowego usunięcia danych bez utraty tysięcy transakcji, które miały miejsce od czasu wykonania ostatniej pełnej kopii zapasowej.

1.1.6 Podsumowanie

Prawidłowa kontrola i konserwacja środowiska bazodanowego to wielowymiarowy proces, który nie ma punktu końcowego, lecz trwa przez cały cykl życia oprogramowania. Złożoność nowoczesnych systemów zarządzania bazami danych wymaga od administratorów i projektantów proaktywnego, a nie jedynie reaktywnego podejścia do pojawiających się problemów.

Samo zaprojektowanie zgodnego ze sztuką schematu relacyjnego jest zaledwie początkiem pracy. Stabilność aplikacji zależy w równej mierze od rygorystycznego zarządzania fizyczną strukturą nośników danych, dogłębnego zrozumienia statystycznych mechanizmów planisty zapytań, a także bezkompromisowego wdrażania zaawansowanych polityk bezpieczeństwa opartego na podziale ról.

Holistyczne spojrzenie na administrację PostgreSQL, obejmujące zarówno automatyzację procesów porządkujących pamięć, jak i ciągle monitorowanie planów wykonania zapytań, pozwala na budowanie rozwiązań prawdziwie skalowalnych. Dzięki zaimplementowaniu odpowiednich strategii prewencyjnych i solidnych mechanizmów odtwarzania po awarii, organizacja zyskuje pewność, że jej kluczowy zasób cyfrowy pozostanie bezpieczny, spójny i gotowy na rosnące obciążenia bez względu na niespodziewane incydenty technologiczne.

1.2 Badania literaturowe

1.2.1 Sprzęt dla bazy danych

1.2.2 Konfiguracja bazy danych PostgreSQL

Wstęp

Rozdział opisuje konfigurację połączenia z bazą danych **PostgreSQL** – dojrzałym, open-source’owym systemem zarządzania relacyjnymi bazami danych (RDBMS), który wyróżnia się pełną zgodnością ze standardem SQL oraz silnym naciskiem na rozszerzalność i niezawodność.

Wymagania:

- dostęp do instancji PostgreSQL,
- zmienne środowiskowe,
- narzędzia projektowe (np. `psql`, `pg_isready`).

Podstawowe parametry

Każde połączenie z PostgreSQL wymaga:

- hosta i portu (domyślnie `localhost` i `5432`),
- nazwy bazy danych,
- użytkownika i hasła.

Dane wrażliwe (np. hasła) przechowuj w **zmiennych środowiskowych** – to oddziela konfigurację od kodu. Pamiętaj: zmienne środowiskowe nie są mechanizmem bezpieczeństwa; w produkcji uzupełnij je o dedykowane zarządzanie sekretami (np. HashiCorp Vault, AWS Secrets Manager).

Lokalizacja i struktura katalogów PostgreSQL

Pliki konfiguracyjne PostgreSQL znajdują się w katalogu danych (`PGDATA`).

Główne pliki konfiguracyjne:

- `postgresql.conf` – podstawowa konfiguracja serwera (port, pamięć, logi)
- `pg_hba.conf` – kontrola dostępu (kto i skąd może się łączyć)
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL

Domyślne lokalizacje:

System | Katalog danych |

```
|-----|-----|Linux (RPM/DEB)| /var/lib/pgsql/data/ lub /var/lib/postgresql/*/main/ |Linux  
(kompilacja źródłowa) | /usr/local/pgsql/data/ | | Windows | C:\Program Files\PostgreSQL\<version>\  
data | | Docker | /var/lib/postgresql/data/ (w kontenerze) |
```

Zmiana katalogu danych:

```
# Inicjalizacja nowego katalogu  
initdb -D /ścieżka/do/katalogu  
  
# Uruchomienie z niestandardowym PGDATA  
pg_ctl -D /ścieżka/do/katalogu start
```

Struktura katalogu danych:

```
$PGDATA/  
├─ postgresql.conf # główny plik konfiguracyjny  
├─ pg_hba.conf     # polityka dostępu  
├─ pg_ident.conf   # mapowanie tożsamości  
├─ base/           # rzeczywiste bazy danych  
└─ global/         # tabele systemowe
```

(continues on next page)

(continued from previous page)

```
└─ log/          # logi serwera
└─ pg_wal/       # Write-Ahead Log (WAL)
```

Sprawdzenie aktualnej lokalizacji:

```
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
```

Środowiska i profile

Przełączanie środowisk PostgreSQL (dev/test/prod) realizuj przez:

- zmienne środowiskowe (np. PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD),
- profile konfiguracyjne,
- osobne pliki .env dla każdego środowiska.

Automatyzacja

Zalecenia dla PostgreSQL:

- walidacja konfiguracji przy starcie za pomocą pg_isready,
- skrypty sprawdzające kompletność zmiennych środowiskowych,
- integracja z CI/CD (np. GitHub Actions, GitLab CI).

Dokumentację generuj automatycznie (Sphinx).

Przykłady**PostgreSQL – zmienne środowiskowe (.env)**

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=aplikacja
PGUSER=app_user
PGPASSWORD=${DB_PASSWORD}
```

PostgreSQL – URL połączenia

```
DATABASE_URL=postgresql://${PGUSER}:${PGPASSWORD}@${PGHOST}:${PGPORT}/${PGDATABASE}
```

PostgreSQL – sprawdzenie połączenia

```
pg_isready -h ${PGHOST} -p ${PGPORT} -U ${PGUSER} -d ${PGDATABASE}
```

Najlepsze praktyki

1. Nie przechowuj sekretów w repozytorium.
2. Stosuj .env.example zamiast rzeczywistych danych.
3. Używaj standardowych zmiennych środowiskowych PostgreSQL (PG*).
4. Waliduj konfigurację przed uruchomieniem (pg_isready).

5. W środowiskach produkcyjnych używaj połączeń SSL/TLS (`sslmode=require`).

Podsumowanie

Poprawna konfiguracja połączenia z PostgreSQL zwiększa bezpieczeństwo i przenośność aplikacji między środowiskami. Standardowe zmienne środowiskowe oraz narzędzia takie jak `pg_isready` ułatwiają automatyzację i walidację.

Szczegółowe omówienie plików konfiguracyjnych

Po omówieniu podstawowych wymagań środowiskowych można przejść do szczegółowej konfiguracji PostgreSQL.

W kolejnych sekcjach przedstawiono najważniejsze parametry konfiguracyjne dostępne w plikach:

- `postgresql.conf` – konfiguracja działania serwera,
- `pg_hba.conf` – kontrola dostępu do bazy danych,
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL.

postgresql.conf

Plik `postgresql.conf` jest głównym plikiem konfiguracyjnym klastra PostgreSQL. Zawiera ustawienia wpływające na działanie serwera, wydajność, bezpieczeństwo, logowanie oraz replikację.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW config_file;
```

Najważniejsze parametry

Połączenia sieciowe

listen_addresses

Określa adresy IP, na których PostgreSQL nasłuchuje połączeń.

Najczęstsze wartości:

- `localhost` – tylko połączenia lokalne,
- `*` – wszystkie interfejsy sieciowe,
- lista adresów, np. `'192.168.1.10,localhost'`.

port

Port TCP używany przez PostgreSQL.

Domyślna wartość:

```
port = 5432
```

max_connections

Maksymalna liczba jednoczesnych połączeń z bazą danych.

Zbyt wysoka wartość może zwiększać zużycie pamięci.

Pamięć i wydajność

shared_buffers

Ilość pamięci RAM używanej przez PostgreSQL do buforowania danych.

Typowe wartości:

- 128MB – środowiska testowe,
- 25% pamięci RAM – środowiska produkcyjne.

work_mem

Ilość pamięci wykorzystywanej przez operacje sortowania i zapytania.

Parametr jest przydzielany dla pojedynczej operacji.

maintenance_work_mem

Pamięć używana podczas operacji administracyjnych, np.:

- VACUUM,
- CREATE INDEX,
- ALTER TABLE.

effective_cache_size

Szacowany rozmiar pamięci podręcznej dostępnej dla PostgreSQL.

Parametr pomaga optymalizatorowi zapytań wybierać odpowiednie plany wykonania.

Logowanie

logging_collector

Włącza zapisywanie logów do plików.

Dostępne wartości:

- on,
- off.

log_directory

Katalog przechowywania logów.

log_filename

Wzorzec nazw plików logów.

Przykład:

```
log_filename = 'postgresql-%Y-%m-%d.log'
```

log_min_duration_statement

Określa minimalny czas wykonania zapytania (w ms), po którym zostanie ono zapisane w logach.

Przykład:

```
log_min_duration_statement = 1000
```

Wartość 1000 oznacza logowanie zapytań trwających dłużej niż 1 sekundę.

Bezpieczeństwo

password_encryption

Algorytm szyfrowania haseł użytkowników.

Zalecana wartość:

```
password_encryption = scram-sha-256
```

ssl

Włącza obsługę połączeń SSL/TLS.

Dostępne wartości:

- on,
- off.

ssl_cert_file / ssl_key_file

Ścieżki do certyfikatu i klucza prywatnego SSL.

Replikacja i WAL

wal_level

Określa poziom szczegółowości Write-Ahead Log (WAL).

Dostępne wartości:

- minimal,
- replica,
- logical.

max_wal_size

Maksymalny rozmiar plików WAL przed wykonaniem checkpointu.

archive_mode

Włącza archiwizację WAL.

archive_command

Polecenie wykonywane podczas archiwizacji plików WAL.

Przykładowa konfiguracja

```
listen_addresses = '*'
port = 5432
max_connections = 200

shared_buffers = 1GB
work_mem = 16MB
maintenance_work_mem = 256MB

logging_collector = on
log_directory = 'log'

ssl = on
password_encryption = scram-sha-256
```

(continues on next page)

(continued from previous page)

```
wal_level = replica
max_wal_size = 2GB
```

Weryfikacja konfiguracji

Po zmianie parametrów konfigurację można przeładować bez restartu serwera:

```
SELECT pg_reload_conf();
```

Niektóre parametry wymagają pełnego restartu PostgreSQL.

Aktualne wartości parametrów można sprawdzić poleceniem:

```
SHOW shared_buffers;
SHOW wal_level;
SHOW max_connections;
```

pg_hba.conf

Plik `pg_hba.conf` (Host-Based Authentication) odpowiada za kontrolę dostępu do serwera PostgreSQL.

Definiuje:

- kto może połączyć się z bazą danych,
- z jakiego adresu IP,
- do jakiej bazy danych,
- przy użyciu jakiej metody uwierzytelniania.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW hba_file;
```

Składnia wpisu

Każdy wpis w `pg_hba.conf` ma następującą strukturę:

TYPE	DATABASE	USER	ADDRESS	METHOD
------	----------	------	---------	--------

Znaczenie pól:

TYPE

Typ połączenia.

Dostępne wartości:

- `local` – połączenia lokalne (Unix socket),
- `host` – połączenia TCP/IP,
- `hostssl` – połączenia TCP/IP z SSL,
- `hostnossl` – połączenia TCP/IP bez SSL.

DATABASE

Nazwa bazy danych, do której użytkownik może się połączyć.

Możliwe wartości:

- konkretna nazwa bazy,
- all – wszystkie bazy,
- sameuser – baza o nazwie zgodnej z użytkownikiem.

USER

Użytkownik lub rola PostgreSQL.

ADDRESS

Adres IP lub zakres sieci w notacji CIDR.

Przykłady:

- 127.0.0.1/32,
- 192.168.1.0/24,
- 0.0.0.0/0 – wszystkie adresy.

METHOD

Metoda uwierzytelniania użytkownika.

Najczęściej używane metody uwierzytelniania

trust

Zezwala na połączenie bez uwierzytelniania.

Zalecane wyłącznie w środowiskach testowych.

reject

Odrzuca połączenie.

md5

Uwierzytelnianie przy użyciu hasła MD5.

Metoda starsza i mniej bezpieczna niż SCRAM.

scram-sha-256

Nowoczesna metoda uwierzytelniania oparta na SCRAM.

Zalecana dla nowych instalacji PostgreSQL.

peer

Uwierzytelnianie na podstawie użytkownika systemowego.

Najczęściej używane dla lokalnych połączeń Linux/Unix.

ident

Mapowanie użytkownika systemowego przy użyciu `pg_ident.conf`.

Przykładowa konfiguracja

```
# Połączenia lokalne
local  all             postgres                    peer

# Połączenia localhost IPv4
host   all             all            127.0.0.1/32    scram-sha-256

# Połączenia localhost IPv6
host   all             all            ::1/128        scram-sha-256
```

(continues on next page)

(continued from previous page)

```
# Sieć lokalna
host      aplikacja      app_user      192.168.1.0/24      scram-sha-256
```

Najlepsze praktyki

1. Używaj `scram-sha-256` zamiast `md5`.
2. Ograniczaj dostęp do konkretnych adresów IP.
3. Nie używaj `trust` w środowiskach produkcyjnych.
4. Stosuj `hostssl` dla połączeń zdalnych.
5. Po zmianie konfiguracji wykonuj reload konfiguracji:

```
SELECT pg_reload_conf();
```

pg_ident.conf

Plik `pg_ident.conf` umożliwia mapowanie użytkowników systemowych na role PostgreSQL.

Najczęściej używany jest razem z metodami uwierzytelniania:

- `peer`,
- `ident`.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW ident_file;
```

Zastosowanie

Mechanizm mapowania pozwala określić, który użytkownik systemowy może zostać powiązany z konkretną rolą PostgreSQL.

Jest to szczególnie przydatne:

- w środowiskach Linux/Unix,
- przy automatyzacji administracyjnej,
- dla lokalnych połączeń bez użycia haseł.

Składnia wpisu

MAPNAME	SYSTEM-USERNAME	PG-USERNAME
---------	-----------------	-------------

Znaczenie pól:

MAPNAME

Nazwa mapowania wykorzystywana w `pg_hba.conf`.

SYSTEM-USERNAME

Użytkownik systemu operacyjnego.

PG-USERNAME

Rola PostgreSQL, na którą użytkownik ma zostać zmapowany.

Przykładowa konfiguracja

# MAPNAME	SYSTEM-USERNAME	PG-USERNAME
admin_map	postgres	postgres
admin_map	deploy	db_admin
app_map	appuser	aplikacja

Przykład użycia w `pg_hba.conf`:

```
local all all peer map=admin_map
```

W powyższym przykładzie użytkownicy systemowi zdefiniowani w `admin_map` mogą logować się jako odpowiadające im role PostgreSQL.

Najlepsze praktyki

1. Ograniczaj mapowania wyłącznie do wymaganych użytkowników.
2. Nie mapuj użytkowników systemowych na role superuser bez uzasadnienia.
3. Stosuj osobne role administracyjne i aplikacyjne.
4. Dokumentuj wszystkie mapowania użytkowników.
5. Regularnie przeglądaj konfigurację dostępu.

1.2.3 Monitorowanie i diagnostyka

Tego typu jakis tekst

1.2.4 Partycjonowanie danych

1.2.5 Bezpieczeństwo

1.2.6 Sprawozdanie z laboratorium documentation

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

Wprowadzenie

Badania literaturowe

Zadania

1.3 Rozdział 3: Projektowanie Bazy Danych: System Zarządzania Biblioteką

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

1.3.1 1. Wybór zagadnienia, opis procesów i danych

Wybrane zagadnienie:

System zarządzania wypożyczeniami w bibliotece. Projekt obejmuje obsługę bazy czytelników, katalogu zbiorów bibliotecznych (z uwzględnieniem autorów i kategorii literackich), a także pełen proces ewidencji wypożyczeń, zwrotów oraz historii aktywności czytelnika.

Opis procesów i więzy integralności:

Głównym procesem w systemie jest obieg książki między biblioteką a czytelnikiem.

- **Rejestracja:** Czytelnik zapisuje się do biblioteki, podając swoje dane osobowe oraz adresowe. System automatycznie rejestruje datę zapisu.
- **Katalogowanie:** Książki są kategoryzowane (np. Fantastyka, Kryminał, Literatura faktu) i przypisane do konkretnych autorów. Każdy fizyczny egzemplarz książki posiada swój unikalny identyfikator.
- **Wypożyczenia i Zwroty:** Proces wypożyczenia łączy czytelnika z konkretnym egzemplarzem książki w czasie. Rejestrowana jest data wypożyczenia. System zakłada konieczność ewidencji daty zwrotu, by w przyszłości móc naliczać ewentualne kary.
- **Statystyki (Więzy integralności):** Pola takie jak “aktualne wypożyczenia” i “suma wypożyczeń” z pierwotnego prototypu są wartościami wyliczalnymi (pochodnymi) na podstawie historii transakcji, co w docelowym modelu gwarantuje spójność danych.

Wykaz gromadzonych danych:

- **Dane czytelnika:** Imię, Nazwisko, Ulica, Kod pocztowy, Miasto, Data zapisu.
- **Dane książki:** Tytuł, Rok wydania, Dostępność.
- **Dane autora:** Imię, Nazwisko.
- **Dane kategorii:** Nazwa gatunku literackiego.
- **Dane transakcyjne:** Data wypożyczenia, Data zwrotu (rzeczywista).

1.3.2 2. Prototyp CSV

Aby zweryfikować kompletność przetwarzanych informacji, przygotowano “płaską” (nieznormalizowaną) reprezentację danych dla operacji wypożyczenia, bazującą na pierwotnych wytycznych.

```
Imie,Nazwisko,Ulica,Kod_Pocztowy,Miasto,Data_Zapisu,Tytul_Ksiazki,Imie_Autora,Nazwisko_Autora,Kategoria,Data_Wypozyczenia,Data_Zwrotu
Jan,Kowalski,Kwiatowa 1,00-001,Warszawa,2024-01-15,Wiedźmin,Andrzej,Sapkowski,Fantastyka,
2024-03-01,2024-03-15
Jan,Kowalski,Kwiatowa 1,00-001,Warszawa,2024-01-15,Pan Tadeusz,Adam,Mickiewicz,Poezja,
2024-03-10,
Anna,Nowak,Polna 2,31-444,Kraków,2023-11-05,Lśnienie,Stephen,King,Horror,2024-02-20,2024-
03-05
```

1.3.3 3. Model Konceptualny (Pojęciowy)

Na podstawie analizy procesów i zebranych danych opracowano model pojęciowy, identyfikując obiekty, ich cechy oraz powiązania.

Zidentyfikowane encje

- **Czytelnik** - osoba zapisana do biblioteki.
- **Autor** - twórca dzieła literackiego.
- **Książka** - oferowana pozycja z inwentarza biblioteki.
- **Kategoria** - sekcja grupująca gatunkowo księgozbiór.
- **Wypożyczenie** - zdarzenie potwierdzające fakt wydania książki czytelnikowi.

Zdefiniowane atrybuty/własności

- **Czytelnik:** Imię, Nazwisko, Ulica, Kod pocztowy, Miasto, Data zapisu.
- **Autor:** Imię, Nazwisko.
- **Książka:** Tytuł, Rok wydania.
- **Kategoria:** Nazwa kategorii.
- **Wypożyczenie:** Data wypożyczenia, Data zwrotu.

Opis związków

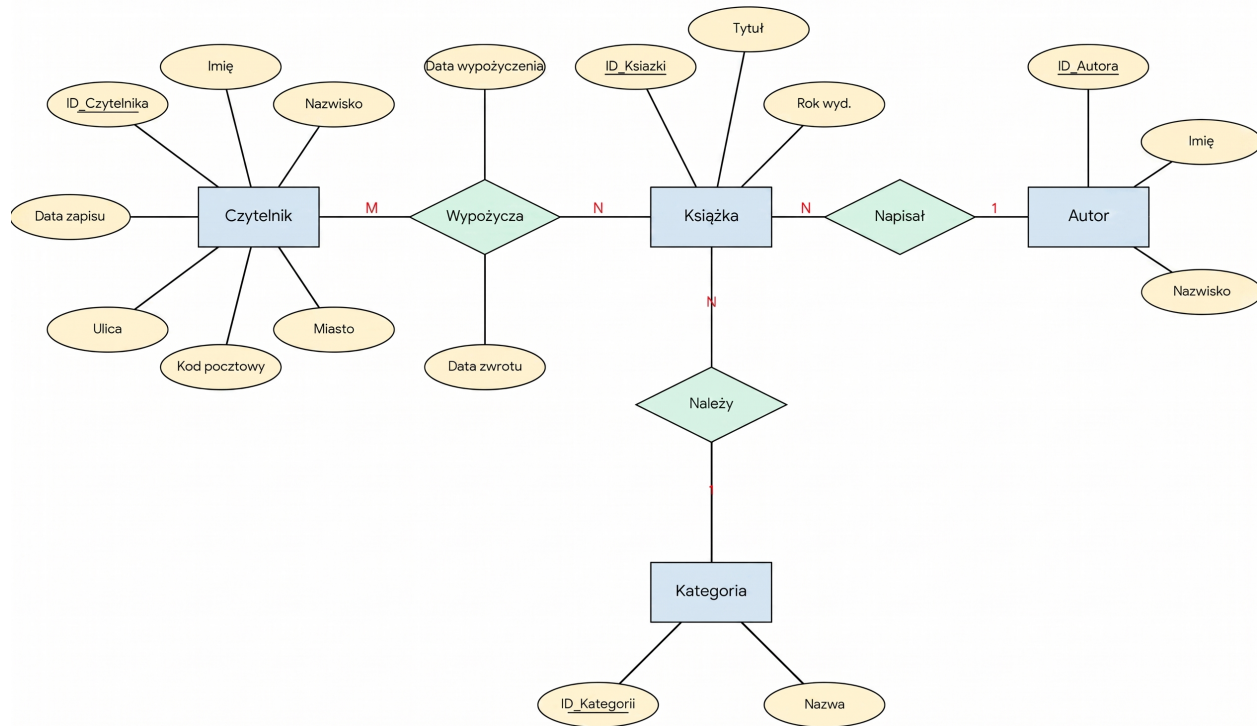
- **Autor – Książka (1:N):** Jeden autor może napisać wiele książek. (Dla uproszczenia modelu zakładamy głównego autora).
- **Kategoria – Książka (1:N):** Kategoria zawiera wiele tytułów.
- **Czytelnik – Wypożyczenie (1:N):** Czytelnik może dokonać wielu wypożyczeń w historii.
- **Książka – Wypożyczenie (1:N):** Jeden egzemplarz książki może być w swojej historii wypożyczany wielokrotnie różnym czytelnikom.

Identyfikacja encji słabych

Występuje relacja wiele-do-wielu (M:N) między Czytelnikiem a Książką (czytelnik czyta wiele książek, książka jest czytana przez wielu czytelników). Związek ten rozwiązywany jest przez encję asocjacyjną (słabą) **Wypożyczenie**. *

Uzasadnienie: Byt ten nie istnieje samodzielnie w oderwaniu od konkretnego Czytelnika i konkretnej Książki.

Schemat w notacji Chena



1.3.4 4. Model logiczny i proces normalizacji

4.1. Przebieg procesu normalizacji

Krok 1: Pierwsza Postać Normalna (1NF) Wiersze są unikalne, tworzymy sztuczne klucze główne (ID_Wypożyczenia). Wartości w komórkach są atomowe. Pojawia się duża redundancja danych adresowych i książkowych.

Krok 2: Druga Postać Normalna (2NF) Rozbijamy płaską strukturę względem częściowych zależności. Oddzielamy encje twarde od operacji. Tworzymy bazowe tabele: Czytelnicy oraz Książki. Powstaje tabela Wypożyczenia przechowująca klucze obce ID_Czytelnika oraz ID_Książki, a także daty transakcji. *Uwaga:* Dynamiczne atrybuty takie jak “suma wypożyczeń” z pierwotnego zadania są usuwane ze schematu, ponieważ łamią zasady normalizacji – można je obliczyć zapytaniem SQL typu COUNT().

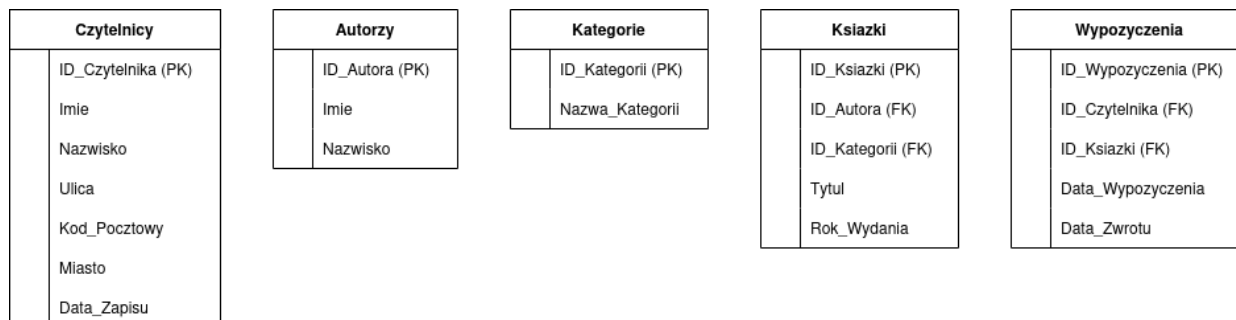
Krok 3: Trzecia Postać Normalna (3NF) Eliminacja zależności przechodnich. Z tabeli Książki wydzielamy powtarzające się nazwy gatunków do tabeli Kategorie (klucz: ID_Kategorii) oraz dane twórców do tabeli Autorzy (klucz: ID_Autora).

4.2. Ostateczna struktura tabel (3NF)

- **Czytelnicy:** ID_Czytelnika (PK), Imię, Nazwisko, Ulica, Kod_Pocztowy, Miasto, Data_Zapisu
- **Autorzy:** ID_Autora (PK), Imię, Nazwisko
- **Kategorie:** ID_Kategorii (PK), Nazwa_Kategorii
- **Książki:** ID_Książki (PK), ID_Autora (FK), ID_Kategorii (FK), Tytuł, Rok_Wydania

- **Wypożyczenia:** ID_Wypożyczenia (PK), ID_Czytelnika (FK), ID_Książki (FK), Data_Wypożyczenia, Data_Zwrotu

4.3. Diagram ERD (Model Logiczny)



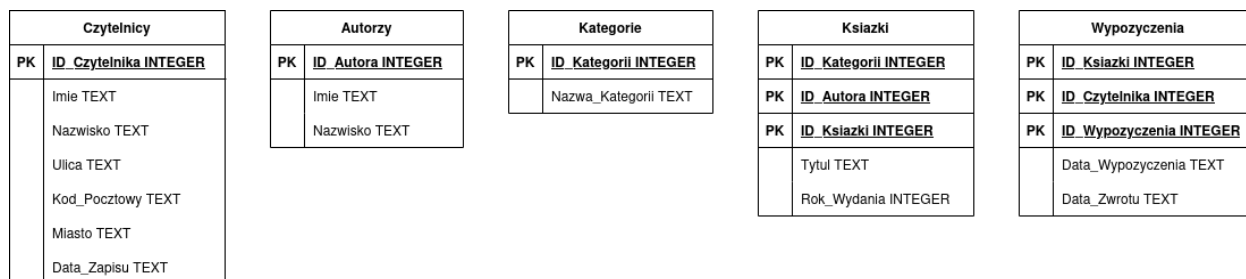
1.3.5 5. Model fizyczny bazy danych

Różnice implementacyjne między modelami fizycznymi wynikają wprost z silników RDBMS – ograniczeń typowania SQLite oraz bogatych możliwości deklaracyjnych PostgreSQL.

5.1. Model fizyczny dla środowiska SQLite

Z uwagi na okrojony zestaw typów, daty są mapowane jako TEXT.

- **Czytelnicy:** ID_Czytelnika : INTEGER PRIMARY KEY, Imie : TEXT, Nazwisko : TEXT, Ulica : TEXT, Kod_Pocztowy : TEXT, Miasto : TEXT, Data_Zapisu : TEXT
- **Autorzy:** ID_Autora : INTEGER PRIMARY KEY, Imie : TEXT, Nazwisko : TEXT
- **Kategorie:** ID_Kategorii : INTEGER PRIMARY KEY, Nazwa_Kategorii : TEXT
- **Książki:** ID_Książki : INTEGER PRIMARY KEY, ID_Autora : INTEGER, ID_Kategorii : INTEGER, Tytul : TEXT, Rok_Wydania : INTEGER
- **Wypożyczenia:** ID_Wypożyczenia : INTEGER PRIMARY KEY, ID_Czytelnika : INTEGER, ID_Książki : INTEGER, Data_Wypożyczenia : TEXT, Data_Zwrotu : TEXT



5.2. Model fizyczny dla środowiska PostgreSQL

PostgreSQL umożliwia zastosowanie precyzyjnych i natywnych typów, w tym rygorystycznych typów daty (DATE) oraz optymalizacji pamięciowej dla ciągów znaków (VARCHAR).

- **Czytelnicy:** ID_Czytelnika : SERIAL PRIMARY KEY, Imie : VARCHAR(50), Nazwisko : VARCHAR(50), Ulica : VARCHAR(100), Kod_Pocztowy : VARCHAR(6), Miasto : VARCHAR(50), Data_Zapisu : DATE DEFAULT CURRENT_DATE
- **Autorzy:** ID_Autora : SERIAL PRIMARY KEY, Imie : VARCHAR(50), Nazwisko : VARCHAR(50)

- **Kategorie:** ID_Kategorii : SERIAL PRIMARY KEY, Nazwa_Kategorii : VARCHAR(50)
- **Ksiazki:** ID_Ksiazki : SERIAL PRIMARY KEY, ID_Autora : INTEGER REFERENCES Autorzy, ID_Kategorii : INTEGER REFERENCES Kategorie, Tytul : VARCHAR(150), Rok_Wydania : SMALLINT
- **Wypozyczenia:** ID_Wypozyczenia : SERIAL PRIMARY KEY, ID_Czytelnika : INTEGER REFERENCES Czytelnicy, ID_Ksiazki : INTEGER REFERENCES Ksiazki, Data_Wypozyczenia : DATE DEFAULT CURRENT_DATE, Data_Zwrotu : DATE

Czytelnicy	
PK	ID_Czytelnika SERIAL
	Imie VARCHAR(50)
	Nazwisko VARCHAR(50)
	Ulica VARCHAR(100)
	Kod_Pocztowy VARCHAR(6)
	Miasto VARCHAR(50)
	Data_Zapisu DATE

Autorzy	
PK	ID_Autora SERIAL
	Imie VARCHAR(50)
	Nazwisko VARCHAR(50)

Kategorie	
PK	ID_Kategorii SERIAL
	Nazwa_Kategorii VARCHAR(50)

Ksiazki	
PK	ID_Kategorii INTEGER
PK	ID_Autora INTEGER
PK	ID_Ksiazki SERIAL
	Tytul VARCHAR(150)
	Rok_Wydania SMALLINT

Wypozyczenia	
PK	ID_Ksiazki INTEGER
PK	ID_Czytelnika INTEGER
PK	ID_Wypozyczenia SERIAL
	Data_Wypozyczenia DATE
	Data_Zwrotu DATE

1.4 Rozdział 4: Implementacja fizyczna i zasilanie bazy danych

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

1.4.1 1. Definicja fizyczna bazy danych (Zadanie 1)

Na podstawie opracowanych wcześniej modeli logicznych przygotowano skrypty DDL (Data Definition Language) dla dwóch docelowych silników relacyjnych baz danych: PostgreSQL oraz SQLite. Kody te posłużyły do utworzenia struktur w bazach dostępnych na serwerze laboratoryjnym oraz przez sieć Internet.

1.1. Skrypt dla środowiska PostgreSQL

Środowisko PostgreSQL pozwala na wykorzystanie zaawansowanych, natywnych typów danych oraz rygorystyczne egzekwowanie więzów integralności. W poniższym skrypcie kluczowym elementem jest użycie pseudo-typu SERIAL dla automatycznej generacji kluczy głównych oraz typów o stałej lub ograniczonej długości (VARCHAR, SMALLINT, DATE) dla optymalizacji przechowywania danych. Relacje między tabelami zapewniane są przez twarde klauzule REFERENCES.

```
-- Fragment kodu definicji kluczowych tabel (PostgreSQL)
CREATE TABLE Ksiazki (
    ID_Ksiazki SERIAL PRIMARY KEY,
    ID_Autora INTEGER REFERENCES Autorzy(ID_Autora),
    ID_Kategorii INTEGER REFERENCES Kategorie(ID_Kategorii),
    Tytul VARCHAR(150) NOT NULL,
    Rok_Wydania SMALLINT
);

CREATE TABLE Wypozyczenia (
    ID_Wypozyczenia SERIAL PRIMARY KEY,
    ID_Czytelnika INTEGER REFERENCES Czytelnicy(ID_Czytelnika),
    ID_Ksiazki INTEGER REFERENCES Ksiazki(ID_Ksiazki),
    Data_Wypozyczenia DATE DEFAULT CURRENT_DATE,
    Data_Zwrotu DATE
);
```

1.2. Skrypt dla środowiska SQLite

W silniku SQLite struktura ulega pewnemu uproszczeniu z powodu mniejszej rygorystyczności typowania oraz bardzo ograniczonej liczby wbudowanych klas typów danych (np. brak odrębnego typu daty). Z tego względu zastosowano typ TEXT dla dat i łańcuchów znaków. Ponadto w SQLite relacje kluczy obcych deklaruje się w osobnej klauzuli FOREIGN KEY na końcu definicji tabeli, a automatyczna inkrementacja klucza realizowana jest atrybutem AUTOINCREMENT.

```
-- Fragment kodu definicji kluczowych tabel (SQLite)
CREATE TABLE Wypozyczenia (
    ID_Wypozyczenia INTEGER PRIMARY KEY AUTOINCREMENT,
    ID_Czytelnika INTEGER,
    ID_Ksiazki INTEGER,
    Data_Wypozyczenia TEXT,
    Data_Zwrotu TEXT,
    FOREIGN KEY(ID_Czytelnika) REFERENCES Czytelnicy(ID_Czytelnika),
    FOREIGN KEY(ID_Ksiazki) REFERENCES Ksiazki(ID_Ksiazki)
);
```

1.4.2 2. Mechanizm zasilania bazy danych (Zadanie 2)

Posiadając gotową strukturę bazy, należało wyposażać ją w dane demonstracyjne. Przygotowano zbiór danych testowych, które zostały poddane wsadowemu ładowaniu (batch import).

2.1. Formatyzacja danych (Pliki CSV)

Dane do importu przygotowano w niezależnym formacie płaskim CSV (Comma-Separated Values). Takie podejście pozwala na łatwą edycję danych poza systemem bazodanowym. Poniżej zaprezentowano wycinek pliku dane_kategorie.csv:

```
Nazwa_Kategorii
Fantastyka
Kryminał
Literatura faktu
Poezja
Horror
```

2.2. Implementacja mechanizmu importu

Do zaimportowania powyższych danych wybrano mechanizm wprowadzania wielowartościowego oparty na technice **INSERT (multi-value/batch)**. Rozwiązanie zrealizowano z wykorzystaniem języka Python oraz dedykowanego sterownika psycopg.

Wybór tego mechanizmu podyktowany jest kompromisem między uniwersalnością a wydajnością. Użycie metody executemany() na obiekcie kursora bazy danych pozwala na szybkie wstawienie wielu rekordów w ramach pojedynczej transakcji sieciowej. Jest to rozwiązanie znacznie bardziej optymalne czasowo i obciążeniowo niż wykonywanie pojedynczych instrukcji INSERT iteracyjnie w pętli. Opcjonalna klauzula COPY byłaby szybsza, jednak rozwiązanie oparte o wsadowy INSERT jest bardziej uniwersalne w architekturze klient-serwer.

Poniżej przedstawiono fragment skryptu aplikacyjnego odpowiedzialnego za odczyt z pliku i zasilenie tabel:

```
import csv
import psycopg

# 1. Wczytywanie danych z pliku CSV do struktury iterowalnej (listy krotek)
def wczytaj_dane_csv(sciezka_pliku):
```

(continues on next page)

(continued from previous page)

```

dane = []
with open(sciezka_pliku, mode='r', encoding='utf-8') as plik:
    csv_reader = csv.reader(plik)
    next(csv_reader) # Pominięcie wiersza nagłówka
    for wiersz in csv_reader:
        dane.append(tuple(wiersz))
return dane

# 2. Właściwy proces wsadowego importu do bazy PostgreSQL
def importuj_do_postgres(kategorie_do_importu, db_config):
    try:
        # Użycie Context Managera dla bezpiecznego zarządzania transakcjami
        with psycopg2.connect(**db_config) as conn:
            with conn.cursor() as cursor:

                # Definicja sparametryzowanego zapytania zapobiegającego SQL Injection
                zapytanie_sql = "INSERT INTO Kategorie (Nazwa_Kategorii) VALUES (%s)"

                # Wsadowe wywołanie zapytań (batch insert)
                cursor.executemany(zapytanie_sql, kategorie_do_importu)

                print(f"Pomyślnie zaimportowano {len(kategorie_do_importu)} rekordów.")

    except Exception as e:
        print(f"Wystąpił błąd operacji wsadowej: {e}")

```

1.4.3 3. Podsumowanie

Niniejszy rozdział wieńczy etap projektowania i wdrażania struktury relacyjnej bazy danych dla systemu zarządzania biblioteką. Poprzez transformację modelu logicznego do postaci fizycznej, udało się z powodzeniem zaimplementować docelowe schematy w dwóch niezależnych środowiskach bazodanowych: produkcyjnym (PostgreSQL) oraz lokalnym/testowym (SQLite). Opracowane skrypty DDL poprawnie uwzględniają specyfikę obu silników, zachowując przy tym pożądaną architekturę relacyjną.

Dodatkowo, proces inicjalizacji systemu został zautomatyzowany dzięki przygotowaniu skryptu w języku Python. Wykorzystanie plików CSV oraz mechanizmu wsadowego wstawiania danych (batch insert) udowodniło, że możliwe jest szybkie i skalowalne zasilenie bazy niezbędnymi danymi demonstracyjnymi. Wdrożone rozwiązania stanowią solidny, zoptymalizowany fundament do dalszych prac nad logiką biznesową i warstwą aplikacyjną całego projektu.